Clase 2 de 5

Google Cloud | The LAB

Arquitectura y diseño

Ingeniería y Análisis de Datos en Google Cloud

UTN · Facultad Regional Rosario

13 de octubre de 2026

El recorrido de los datos

La clase sigue a los datos desde que entran al sistema hasta que alguien los consulta. En cada etapa hay una decisión de diseño que alguien tiene que tomar, y hoy las tomamos nosotros.

1

Ingesta

Con qué forma entran los datos, y quién declara sus tipos.

2

Almacenamiento y costo

Cómo quedan almacenados y cuánto cuesta leerlos.

3

Partitioning y clustering

Cómo se organizan las filas, según cómo se las va a filtrar.

Lo que vimos en la clase 1

La fuente no se consulta

Guarda el registro entero, compite con la aplicación y no conserva la historia.

Los datos se replican a un data warehouse

Un sistema construido para responder preguntas sobre millones de registros.

Almacenamiento y cómputo separados

El cómputo se asigna por query y se paga el uso.

El proyecto, unidad de facturación y de permisos

Lo que se otorga arriba se hereda: proyecto, dataset, tabla.

Hoy · abrimos el data warehouse

Con qué forma entran los datos, cómo quedan almacenados y cuánto cuesta leerlos, y cómo se organizan las filas.

1

Ingesta

Con qué forma entran los datos

2

Almacenamiento y costo

Cómo quedan almacenados y cuánto cuesta leerlos

3

Partitioning y clustering

Cómo se organizan las filas

Etapa 1 · La ingesta de los datos

Con qué métodos y en qué formatos llegan los datos, quién declara los tipos de una tabla, y por qué los datos crudos se guardan sin transformar.

Cómo se cargan datos en BigQuery

Cuatro métodos, distinguidos por quién mueve los datos y con qué cadencia. La elección entre ellos es la primera decisión de arquitectura de una plataforma.

Método	Qué ocurre	Casos
Se copia un archivo	Un trabajo de carga lee un archivo y lo deja como tabla	Consola, bq load, API. Formatos CSV, JSON, Avro, Parquet
Lo escribe un proceso	Un sistema externo inserta de forma continua o programada	Storage Write API, Pub/Sub, Datastream para CDC, Data Transfer Service, Dataflow, Airbyte o dlt por API
No se mueve	Los datos se consultan donde están, sin copia previa	Tabla externa sobre Cloud Storage, query federada a Cloud SQL o Spanner
Nace adentro	Una tabla se construye consultando otra ya existente	CREATE TABLE AS SELECT, y todo lo que produce dbt

Formatos de archivo

El formato decide si los tipos llegan con los datos o hay que declararlos, y qué tan rápido se puede cargar el archivo.

CSV

```
vendor,pickup,tarifa
1,2022-01-10 08:15,12.5
2,10/01/2022 08:20,9.0
1,2022-01-10 08:31,21.0
2,2022-01-10 08:44,7.5
```

Sin tipos: todo es texto

Comprimido, se descomprime entero antes de repartir la carga

Exportaciones de planillas, sistemas contables y ERPs

JSON

```
{"tarifa":12.5,"pago":"tarjeta"}
{"tarifa":9.0,"pago":"efectivo"}
{"tarifa":7.5,"peaje":{"usd":6}}
```

Un registro por línea; admite anidados

Tipos en parte: las fechas llegan como texto

Comprimido, igual que CSV

Respuestas de APIs, webhooks y logs

Avro

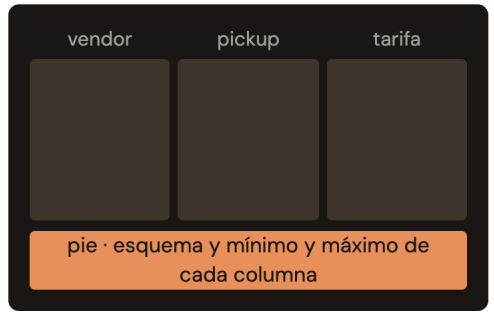


Tipos en el esquema del archivo

Se reparte entre slots aun comprimido: el más rápido para cargar

Exportaciones de bases de datos y pipelines de streaming, como Kafka

Parquet



Tipos en el esquema del archivo

Columnar, como el almacenamiento de BigQuery

Data lakes, procesos con Spark y archivos analíticos en Cloud Storage

El archivo del TP es un CSV: no trae tipos, así que se declaran. Lo confiable se tipa al cargar; lo ambiguo queda como texto y se convierte en la capa siguiente.

Los tipos de datos

Cada columna tiene un tipo, y el tipo define qué valores acepta y qué operaciones se pueden hacer con ella.

Tipo	Qué guarda	Ejemplo
STRING	Texto	Manhattan
INTEGER · INT64	Números enteros	132
FLOAT · FLOAT64	Números con decimales, aproximados	2.75
NUMERIC	Números con decimales, exactos	18.40
BOOLEAN · BOOL	Verdadero o falso	true
DATE	Una fecha	2026-09-14
DATETIME	Fecha y hora, sin zona horaria	2026-09-14 08:30:00
TIMESTAMP	Un instante en el tiempo, guardado en UTC	2026-09-14 11:30:00 UTC

En el esquema se escribe el primer nombre; en SQL, el segundo. Son el mismo tipo.

Para particionar por una columna, tiene que ser DATE, DATETIME, TIMESTAMP o INTEGER.

El esquema de una tabla

El esquema es la definición de una tabla: qué columnas tiene, de qué tipo es cada una y si puede venir vacía. BigQuery lo necesita para poder crear la tabla. Si no se lo pasás escrito, lo deduce solo, mirando unas filas del archivo.

Así se escribe

```
[
  {"name": "pedido_id",
   "type": "INTEGER",
   "mode": "REQUIRED"},
  {"name": "monto",
   "type": "STRING",
   "mode": "NULLABLE"}
]
```

Un objeto por columna, en un archivo **.json** que se le pasa al trabajo de carga.

Lo que trae el archivo	Lo decide el sistema	Lo decidís vos
05/01/2022	STRING. No sabe si 05 es el día o el mes	STRING, porque el formato es ambiguo, y la fecha se arma en la capa siguiente
1.234,56	FLOAT, y guarda 1.23456: el valor queda mil veces más chico, sin aviso	STRING, y se normaliza una sola vez, en un lugar visible
Una columna vacía en las primeras filas	Adivina el tipo mirando una muestra, y puede errarle	El tipo lo elige quien diseña, no la muestra
Una columna nueva que agregó el origen	Entra sola y nadie se entera	La carga falla y alguien tiene que decidir qué hacer

Que la carga se rompa es preferible a que siga andando sobre datos que nadie revisó.

Escrito como archivo, el esquema se versiona en el repositorio como el código.

Los datos crudos se cargan sin transformar

La capa cruda es una copia fiel del origen: no filtra ni descarta. Se tipan las columnas confiables; las de formato ambiguo quedan como texto y se convierten en la capa siguiente.

1 · Cruda · bronze

Idéntica al archivo

Sin filtrar, sin descartar, sin corregir valores.

→

2 · Curada · silver

Tipos y reglas

Fechas, números y criterios de descarte, escritos en código y con pruebas.

→

3 · Consumo · gold

Métricas acordadas

Una definición por métrica, lista para consultar.

Qué se gana

Si una regla estaba mal, se corrige y las capas superiores se reconstruyen desde el crudo.

Qué se paga

Los datos se almacenan dos veces.

Recorrido · cargar un CSV

Crear el dataset donde va a vivir el crudo

Cargar el archivo con autodetección y ver qué adivina

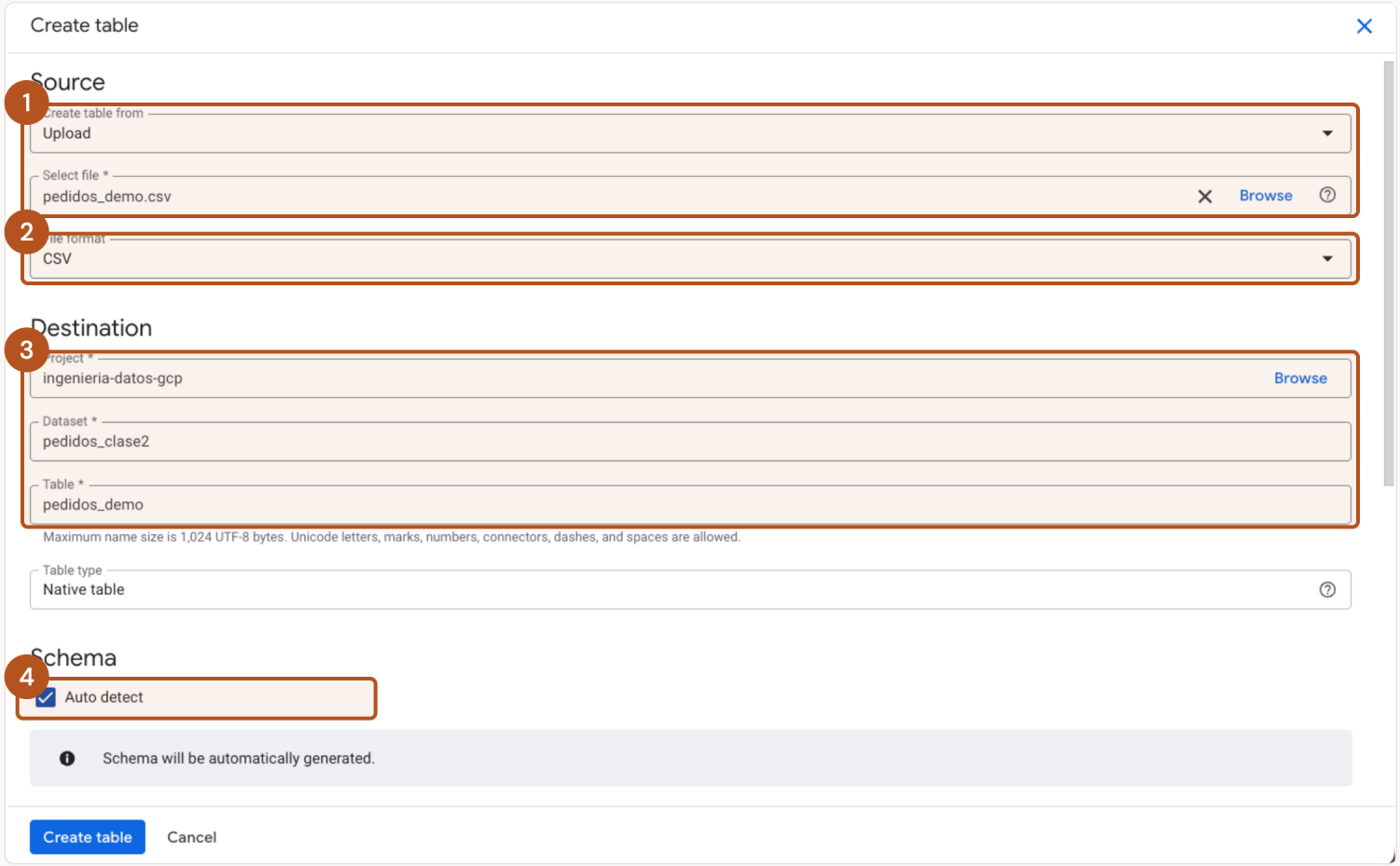
Cargarlo otra vez con el esquema declarado

Comparar lo que adivinó con lo que quedó declarado

Google Cloud | The LAD

Recorrido de carga · paso 1

Cargar el CSV con autodetección

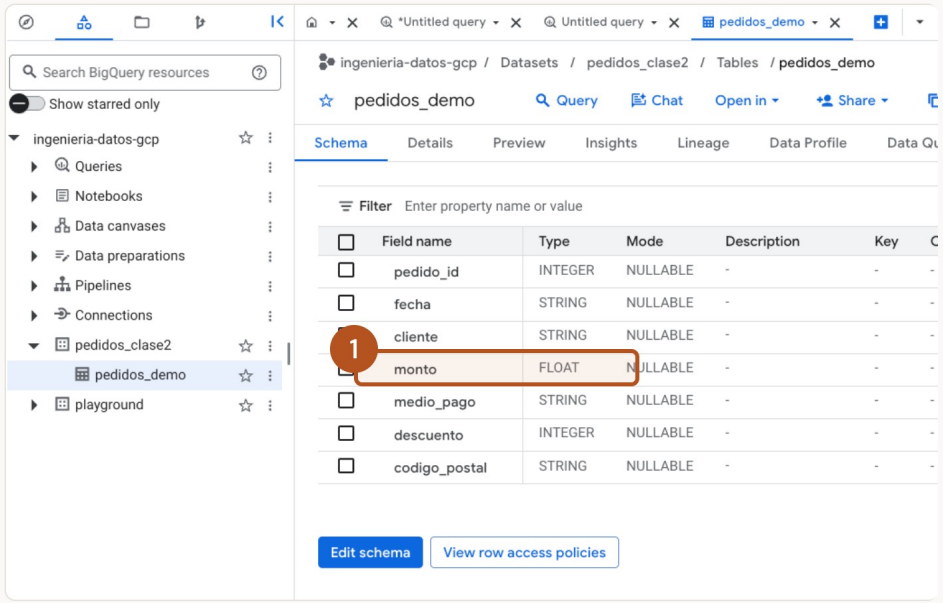


1. **Origen**
   Se sube el archivo desde la computadora.
2. **Formato**
   CSV.
3. **Destino**
   Proyecto, dataset y nombre de la tabla.
4. **Esquema**
   Con la autodetección marcada, el sistema decide los tipos.

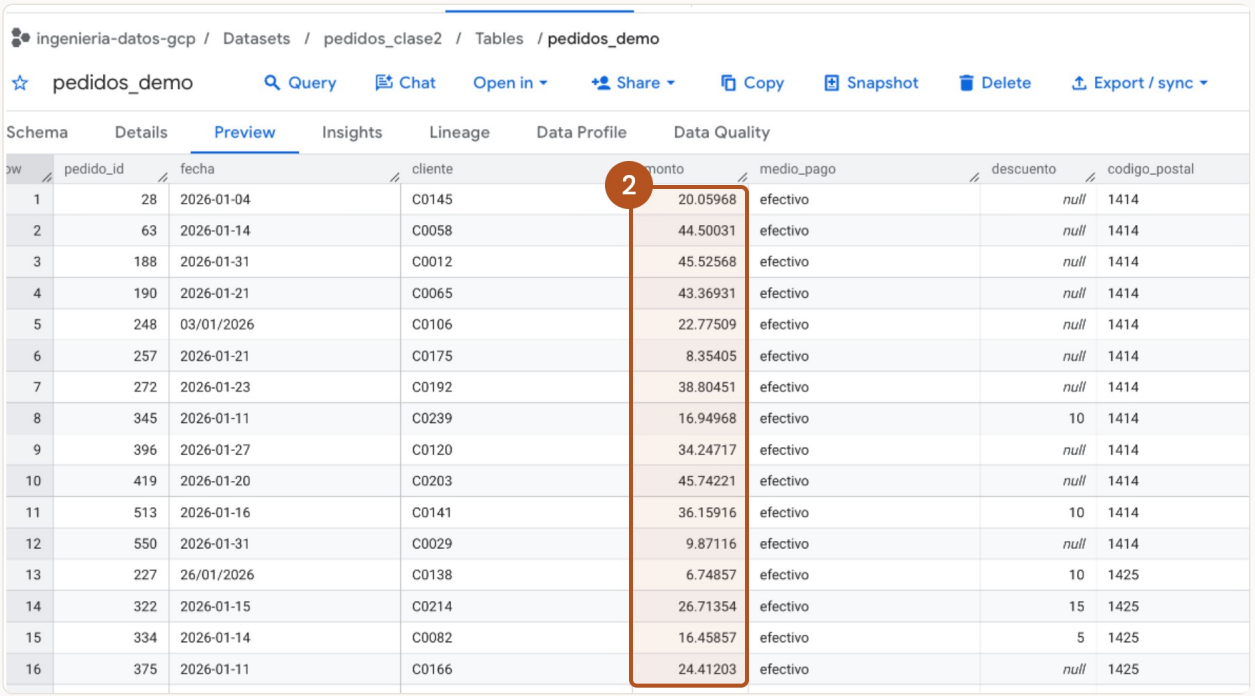
Recorrido de carga · paso 2

La carga terminó bien, y los montos quedaron mal

Esquema



Vista previa



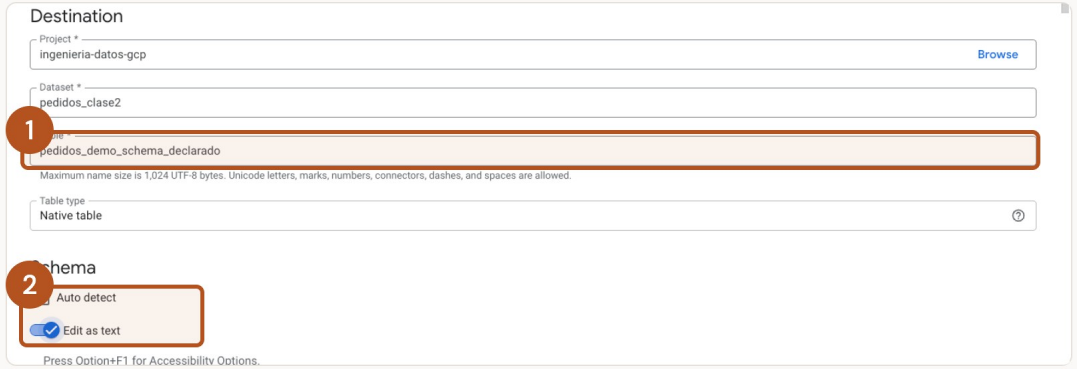
pedido_id	fecha	cliente	monto	medio_pago	descuento	codigo_postal
28	2026-01-04	C0145	20.05968	efectivo	null	1414
63	2026-01-14	C0058	44.50031	efectivo	null	1414
188	2026-01-31	C0012	45.52568	efectivo	null	1414
190	2026-01-21	C0065	43.36931	efectivo	null	1414
248	03/01/2026	C0106	22.77509	efectivo	null	1414
257	2026-01-21	C0175	8.35405	efectivo	null	1414
272	2026-01-23	C0192	38.80451	efectivo	null	1414
345	2026-01-11	C0239	16.94968	efectivo	10	1414
396	2026-01-27	C0120	34.24717	efectivo	null	1414
419	2026-01-20	C0203	45.74221	efectivo	null	1414
513	2026-01-16	C0141	36.15916	efectivo	10	1414
550	2026-01-31	C0029	9.87116	efectivo	null	1414
227	26/01/2026	C0138	6.74857	efectivo	10	1425
322	2026-01-15	C0214	26.71354	efectivo	15	1425
334	2026-01-14	C0082	16.45857	efectivo	5	1425
375	2026-01-11	C0166	24.41203	efectivo	null	1425

1 El monto quedó como FLOAT.

2 El pedido 28 dice 20.059,68 en el archivo y 20.05968 en la tabla. BigQuery lee los números con la convención estadounidense: el punto es el decimal y la coma separa miles.

Recorrido de carga · paso 3

Cargar con el esquema declarado





1 Otra tabla

Con otro nombre, para comparar las dos.

2 Sin autodetección

El esquema se pega como texto: es el archivo .json del repositorio.

3 pedido_id: INTEGER

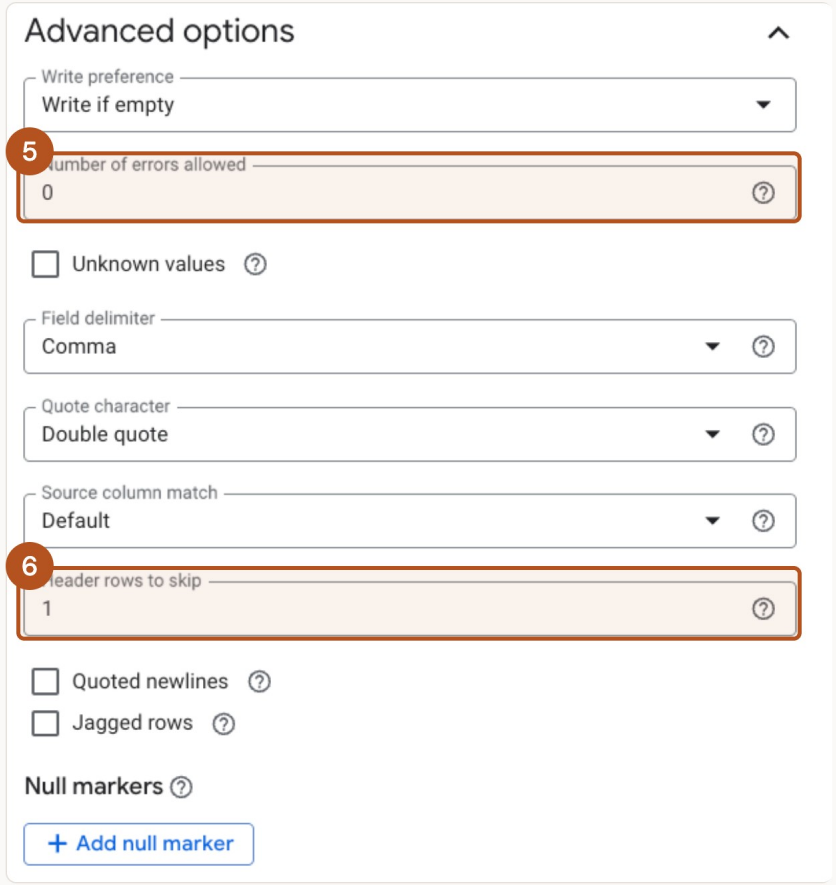
Siempre es un entero. Es confiable, se tipa.

4 monto: STRING

Viene en convención argentina. Es ambiguo, queda como texto.

Recorrido de carga · paso 3

Lo que la autodetección hacía sola



5 Errores permitidos: 0

Una sola fila que no cumpla el esquema hace fallar la carga entera. Es el valor por defecto, y conviene dejarlo así.

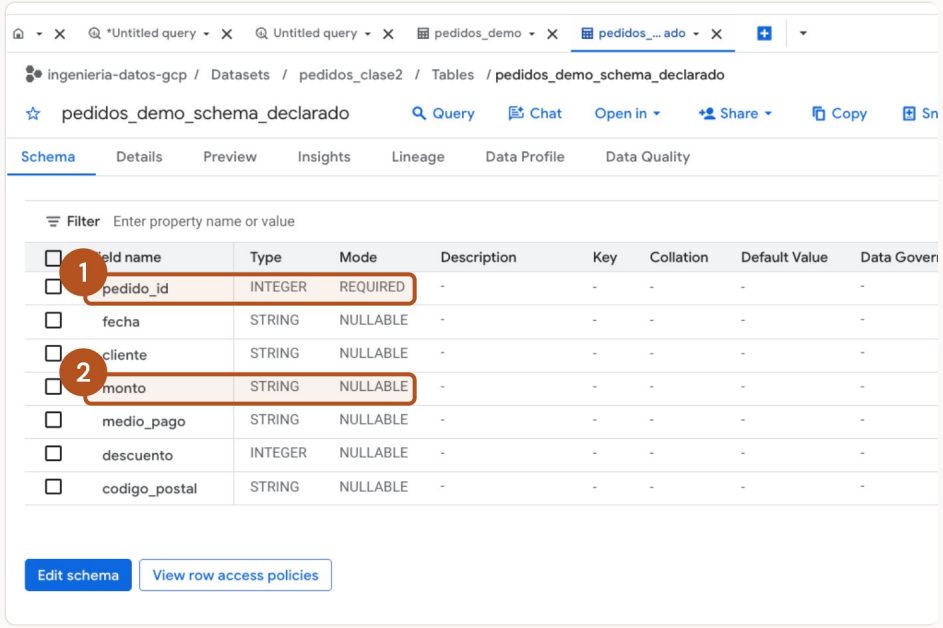
6 Filas de encabezado a saltear: 1

La autodetección reconoce la fila de títulos sola. Con el esquema declarado hay que indicarla: si no, intenta cargar "pedido_id" como entero y la carga falla.

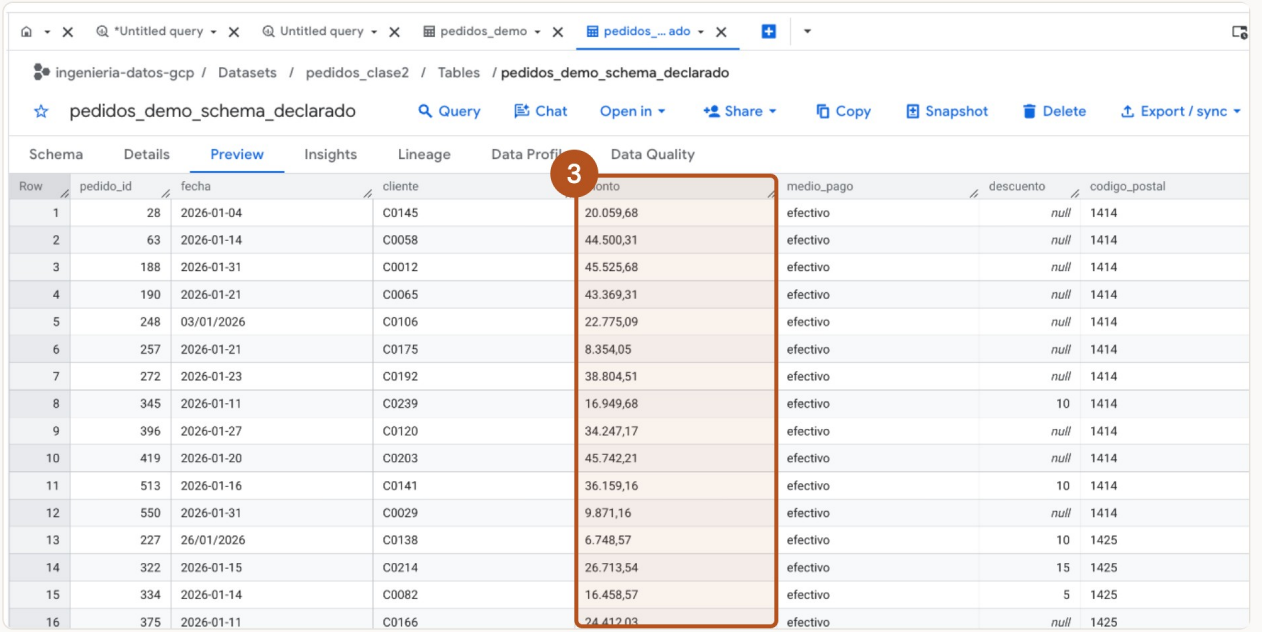
Recorrido de carga · paso 4

Los valores quedaron como en el archivo

Esquema



Vista previa



1 · 2 Cada columna tiene el tipo que se decidió, no el que se adivinó.

3 El pedido 28 dice 20.059,68, igual que en el archivo. La conversión a número se hace en la capa siguiente.

1

Ingesta

Con qué forma entran los datos

2

Almacenamiento y costo

Cómo quedan almacenados y cuánto cuesta leerlos

3

Partitioning y clustering

Cómo se organizan las filas

Etapa 2 · El almacenamiento y el costo

Cómo está construido BigQuery, cómo y dónde guarda los datos, cómo se recupera lo que se borró, cómo se contrata el cómputo y cuánto cuesta una query.

La arquitectura de BigQuery

El almacenamiento y el cómputo son dos sistemas separados, unidos por la red interna de Google. En una base tradicional los dos viven en la misma máquina, y crecer en datos obliga a crecer en máquina.

Cómputo · Dremel

slot	slot	slot	slot	slot	+ n

Se asignan cuando se ejecuta la query y se liberan al terminar. Una query más pesada recibe más slots, sin que nadie dimensione nada.

Jupiter · la red interna, del orden de petabits por segundo

Almacenamiento · Colossus

columna	columna	columna	columna	columna	columna

Los datos quedan quietos, guardados por columna y comprimidos. Siguen ahí cuando no hay ninguna query corriendo.

Se factura por separado

El almacenamiento se cobra por lo guardado. El cómputo, por bytes leídos o por slots, según el modelo contratado.

Escalan por separado

Se pueden guardar años de histórico sin agregar capacidad de cómputo.

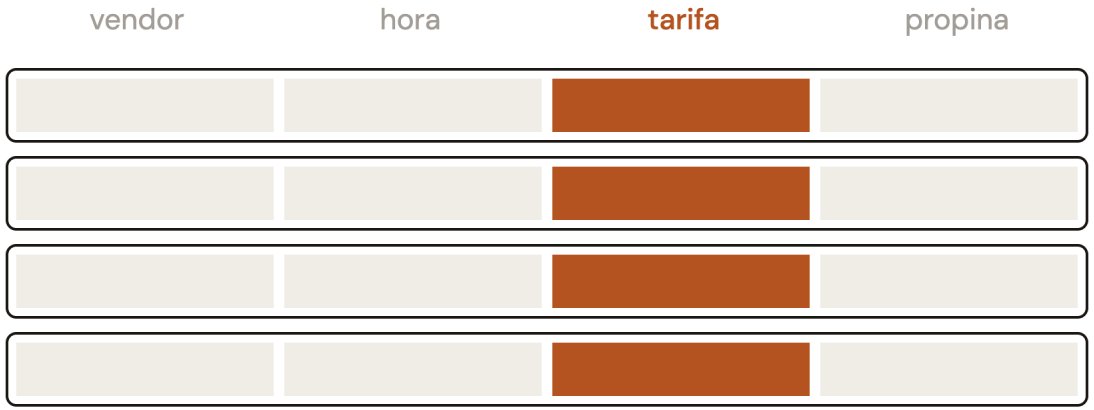
Los límites se configuran

BigQuery no frena por sí solo una query cara. El tope de bytes no viene activado y la cuota diaria arranca en 200 TiB.

Por qué guarda por columnas

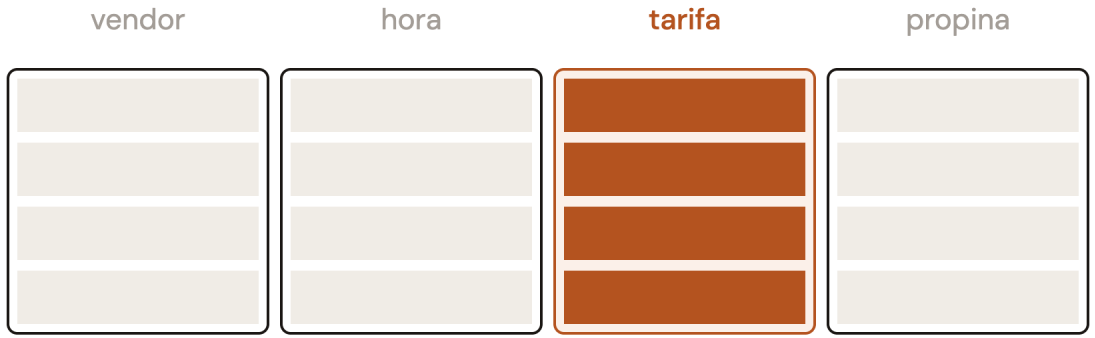
Los mismos cuatro viajes, guardados de dos maneras. La query es la misma: el promedio de la tarifa.

Por filas



16 valores leídos para usar 4

Por columnas

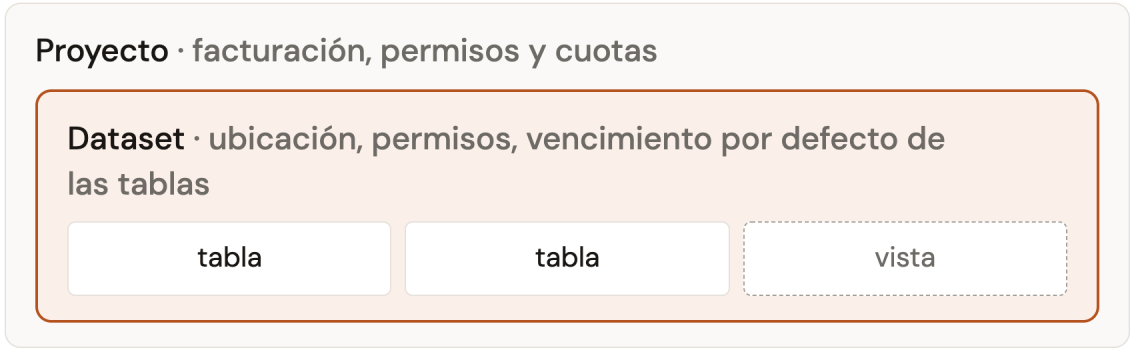


4 valores leídos para usar 4

El recuadro negro es lo que el sistema tiene que abrir entero. Lo que se paga no depende de cuántas columnas tenga la tabla, sino de cuántas pida la query.

Dónde viven los datos

Todo en BigQuery pertenece a un proyecto, que es la unidad de facturación y de permisos. Las tablas se agrupan en datasets, y cada dataset tiene una ubicación que se elige al crearlo.



La ubicación del dataset

Una región, como southamerica-east1 en San Pablo, o una multirregión, como US o EU

No se puede cambiar después de crear el dataset: hay que copiarlo o recrearlo

Una query solo puede leer tablas que estén en la misma ubicación

Para cruzar las tablas propias con las públicas de taxis, el dataset tiene que estar en la misma ubicación que ellas: la multirregión US.

Recuperar lo que se borró

BigQuery conserva el historial de cada tabla durante una ventana de time travel. Dentro de esa ventana se puede consultar la tabla como estaba en un momento anterior, o recuperarla si se borró.

Desde que una tabla se borra o se modifica

Time travel · hasta 7 días	Fail-safe · 7 días más	Después
Se consulta como estaba y se recupera, por cuenta propia	No se consulta: solo la recupera el soporte de Google	Ya no se puede recuperar

Consultar cómo estaba hace una hora

```
SELECT *
FROM curso.viajes
  FOR SYSTEM_TIME AS OF
  TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)
```

Recuperar una tabla borrada

```
bq cp 'curso.viajes@-3600000' \
  curso.viajes_recuperada
```

Copia la tabla como estaba una hora antes: el número son milisegundos hacia atrás.

La ventana se configura por dataset, entre 2 y 7 días. Con facturación lógica, la de por defecto, el historial va incluido en el precio del almacenamiento.

Los dos modelos de contratación

El cómputo se contrata de una de dos maneras, y la elección es una decisión de arquitectura: cambia qué se mide, qué se puede prever y qué se puede limitar.

On demand

Se paga por bytes leídos

USD 6,25 por TiB procesado

Hasta unos **2.000 slots** simultáneos por proyecto, compartidos entre todas sus queries

No hay nada que contratar ni dimensionar

Conviene con uso bajo o variable: se paga solo lo que se consulta, y los meses sin actividad no cuestan.

Capacidad · Editions

Se paga por slot-hora

Tres ediciones: **Standard, Enterprise y Enterprise Plus**

Del orden de **USD 0,04 a 0,10** por slot-hora

La reserva escala sola hasta un **máximo que se fija**

Compromisos a uno o tres años, más baratos, en bloques de 50 slots

Conviene con uso alto y sostenido, cuando el gasto tiene que ser previsible, o para aislar cargas entre equipos.

Almacenamiento

Se factura aparte e igual en los dos modelos: USD 23,55 por TiB al mes. Lo que no se modifica en 90 días baja a USD 16,38 por TiB.

Free tier

Todos los meses, por cuenta de facturación: 1 TiB de queries on demand y 10 GiB de almacenamiento sin cargo.

Sandbox · donde trabaja el curso

Sin tarjeta ni cuenta de facturación. El mismo TiB mensual, 10 GiB de almacenamiento, tablas que vencen a los 60 días y sin streaming ni DML.

Lo que cuesta leer, en modo on demand

La tabla pública de viajes de 2022: 36.256.539 filas y 24 columnas, 6,97 GB. La misma pregunta, escrita de cuatro maneras. En el modelo de capacidad estos bytes no se facturan: se paga el tiempo de los slots.

SELECT *

6,97 GB

Lee las 24 columnas, aunque la query use dos.

Nombrando dos columnas

830 MB

Más de ocho veces menos, para contestar lo mismo.

Agregando LIMIT 10

6,97 GB

El límite recorta el resultado, no la lectura.

La misma query, otra vez

0 B

Sale del caché, que dura 24 horas y no se cobra.

Un tablero que repite esa query cada cinco minutos sobre una tabla que se actualiza	Leído por mes	Sobre el TiB libre	Factura mensual
Nombrando las dos columnas	6,8 TiB	5,8 TiB	USD 36
Con SELECT *	58,8 TiB	57,8 TiB	USD 361

El caché es por usuario y por proyecto: si diez personas abren el mismo tablero, cada una paga su primera ejecución. Y cualquier cambio en la tabla lo invalida.

Recorrido · medir antes de correr

El estimador de bytes, dónde está y qué dice

La misma query con **SELECT *** y con dos columnas

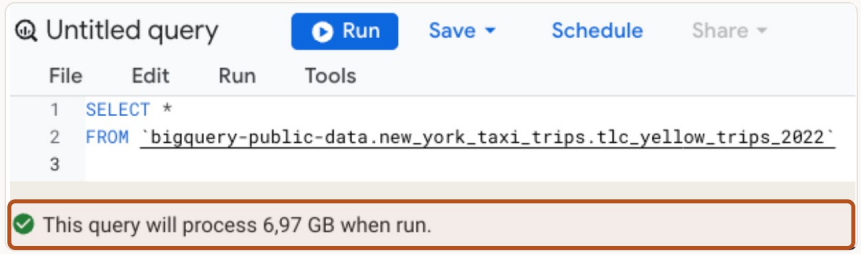
Qué pasa al agregar **LIMIT 10**

Lo que se facturó al ejecutar

Google Cloud | The LAD

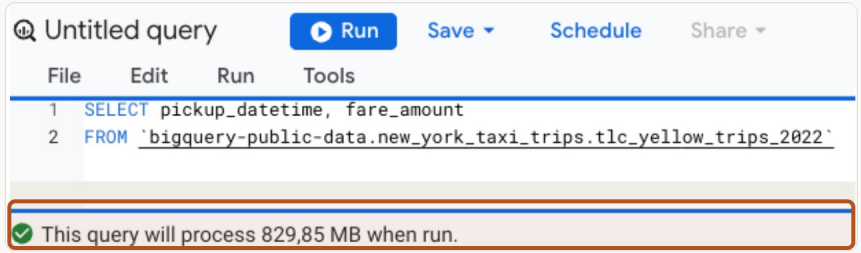
Recorrido de bytes · paso 1

El estimador, antes de ejecutar



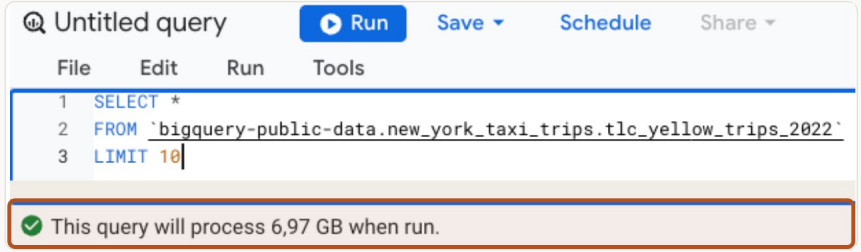
6,97 GB

Lee las 24 columnas.



830 MB

Más de ocho veces menos, para responder lo mismo.



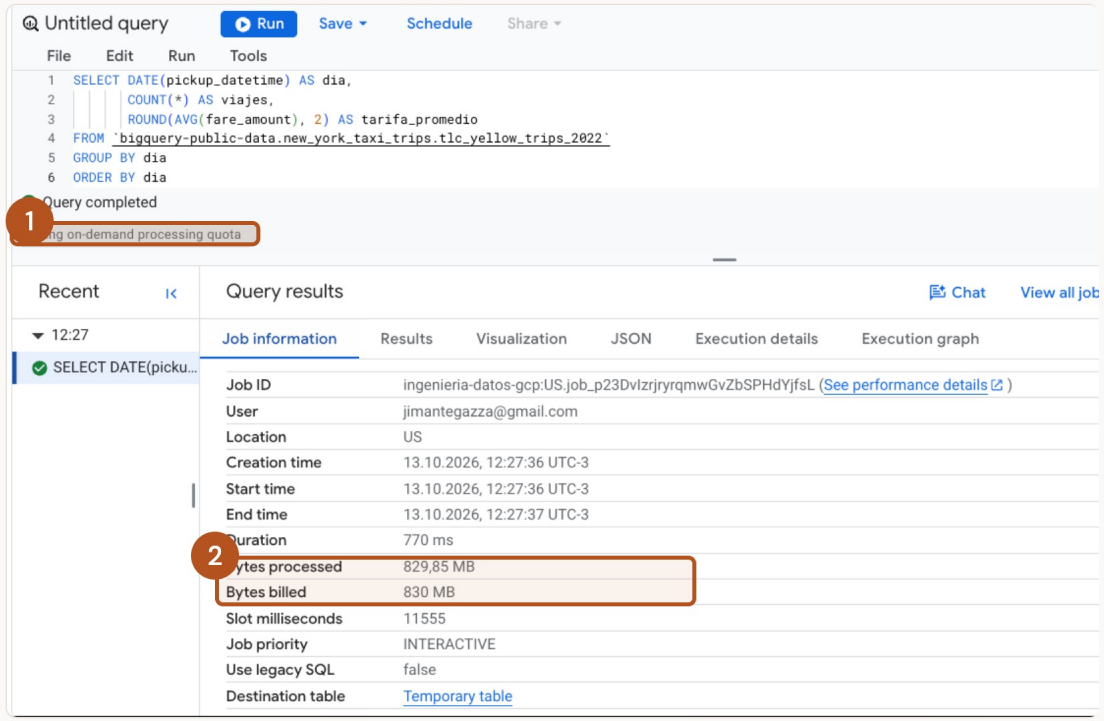
6,97 GB

El LIMIT recorta el resultado, no la lectura.

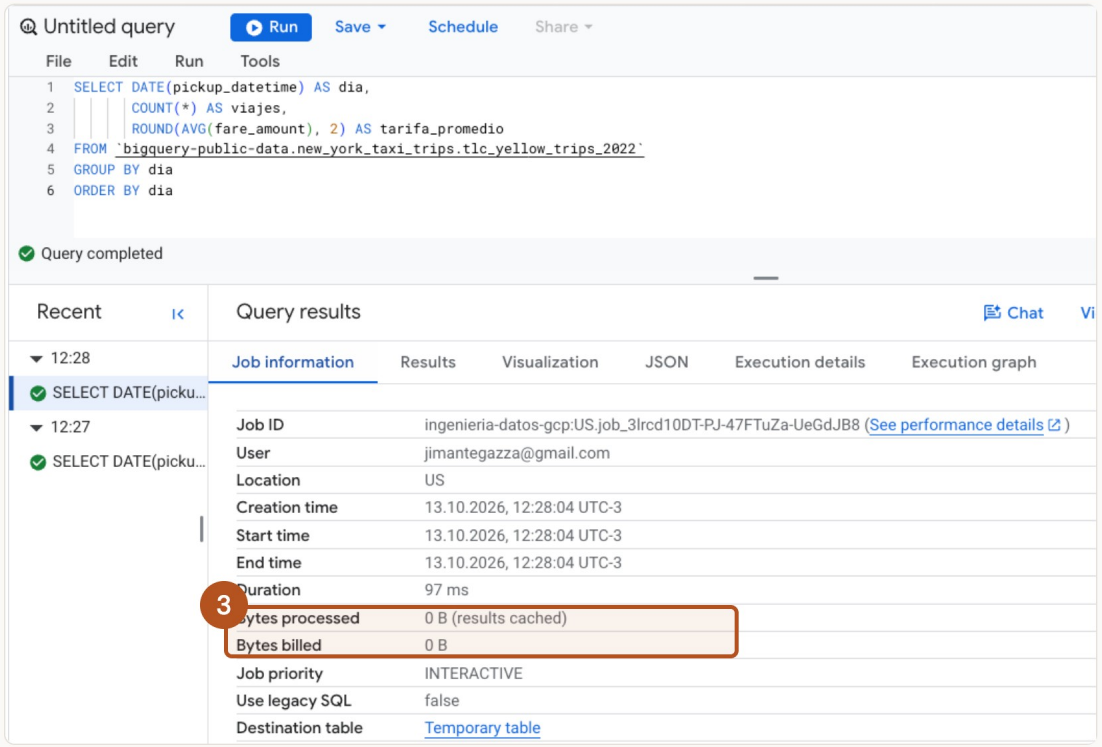
Recorrido de bytes · paso 2

Ejecutar y mirar lo que se facturó

La query de dos columnas, agregada por día



La misma query, otra vez



1 Modo on demand: se factura por bytes leídos.

2 830 MB facturados: lo que había anticipado el estimador.

3 Sale del caché: 0 B facturados.

10:00

Receso

Etapa 3 · Partitioning y clustering

Cómo se organiza una tabla para que una query lea solo las filas que necesita, con qué criterio se eligen las columnas de partición y de clustering, y cuándo no conviene usar ninguna de las dos.

Partitioning

Una tabla particionada se divide en segmentos, llamados particiones, según el valor de una columna. Cada partición se guarda y se administra por separado, y una query que filtra por esa columna lee solo las particiones que corresponden.

Un mes particionado por día · la query pide del 10 al 12 · se leen 3 particiones de 31



Cómo se declara

```
CREATE TABLE curso.viajes
PARTITION BY DATE(pickup_datetime)
AS
SELECT pickup_datetime, pickup_location_id, fare_amount
FROM curso.viajes_crudo
WHERE pickup_datetime >= '2026-09-01'
```

Tipos de partitioning

Una fecha u hora
por hora, día, mes o año

El momento de la carga
sin columna propia

Un número entero
en rangos fijos

Además de leer menos, cada partición vence, se borra y pasa a precio de largo plazo por separado del resto de la tabla.

Cuándo una query aprovecha la partición

Antes de leer, BigQuery usa el WHERE para descartar las particiones que no pueden contener filas del resultado. Ese descarte se llama **partition pruning**, y ocurre solo si el filtro está sobre la columna de partición y la compara con un valor que se conozca sin leer datos.

Lee solo las particiones del filtro

```
pickup_datetime >= '2026-09-21'
AND pickup_datetime < '2026-09-24'
```
Rango sobre la columna de partición

```
pickup_datetime > CURRENT_TIMESTAMP() - INTERVAL 7 DAY
```
CURRENT_TIMESTAMP cuenta como constante

```
TIMESTAMP_TRUNC(pickup_datetime, MONTH) = '2026-09-01'
```
Función admitida, con argumentos constantes

Lee todas las particiones

```
pickup_location_id = '132'
```
Filtra por otra columna

```
EXTRACT(MONTH FROM pickup_datetime) = 9
```
Función que impide el descarte

```
pickup_datetime = (SELECT MAX(...) FROM ...)
```
El valor sale de una subquery

```
pickup_datetime >= '2026-09-21'
OR pickup_location_id = '132'
```
OR con una columna que no es la de partición

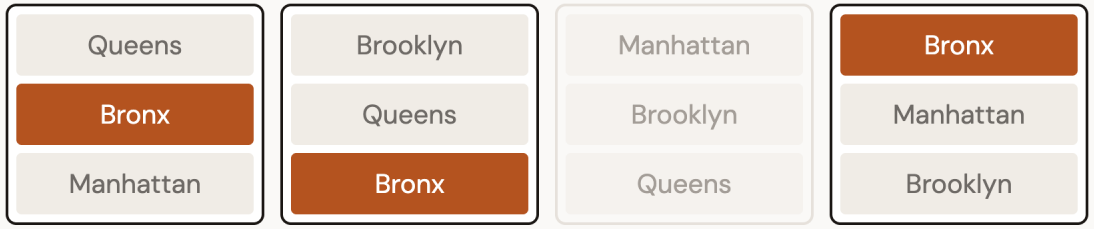
El estimador lo confirma. El descarte se refleja antes de ejecutar: si el número no baja, el filtro no está descartando particiones.

Se puede exigir. Con require_partition_filter, la tabla rechaza toda query que no filtre por la columna de partición.

Clustering

Una tabla con clustering guarda sus filas ordenadas según las columnas elegidas, y cada bloque de almacenamiento conserva metadatos sobre los valores que contiene. Con eso, una query que filtra por esas columnas saltea los bloques que no pueden tener filas del resultado: es el **block pruning**.

Sin clustering · la query pide Bronx · se leen 3 bloques de 4



Con clustering por barrio · se lee 1 bloque de 4



Cómo se declara

```
CREATE TABLE curso.viajes
PARTITION BY DATE(pickup_datetime)
CLUSTER BY pickup_location_id
AS SELECT ...
```

Hasta cuatro columnas. El orden en que se declaran es el orden en que se ordena la tabla.

Cómo se filtra

- Por la primera columna del CLUSTER BY, que es la que más descarta
- Con la columna sola, sin funciones encima
- Contra un valor, no contra otra columna

Qué cambia respecto del partitioning

- Admite columnas con muchos valores distintos
- El estimador no anticipa el ahorro: aparece en los bytes facturados
- BigQuery mantiene el orden solo, sin costo

Partitioning y clustering, lado a lado

	Partitioning	Clustering
Qué hace	Divide la tabla en segmentos que se administran por separado	Ordena el almacenamiento para que cada bloque sea homogéneo
Columnas	Una sola: fecha u hora, un entero en rangos, o el momento de la carga	Hasta cuatro, y el orden en que se declaran importa
Valores distintos	Pocos: cada partición debería reunir un volumen considerable	Muchos: es donde más rinde
Cuándo se ve el ahorro	Antes de ejecutar: el estimador lo descuenta	Después de ejecutar: en los bytes facturados
Qué más permite	Vencer, borrar o reemplazar datos por partición, y precio de largo plazo por partición	Nada más: BigQuery mantiene el orden solo y sin costo

Se combinan: primero se divide la tabla en particiones y después se ordena cada una. Conviene declarar las dos al crear la tabla: el partitioning no se puede cambiar después, y el clustering agregado después no reordena los datos existentes.

Con qué criterio se elige

La guía de Google parte del clustering y agrega partitioning cuando hace falta algo que el clustering no da. Es una guía, no una regla: la decisión sale de cómo se va a filtrar la tabla.

Clustering

Primera opción, según Google

- Una tabla sin particionar de más de 64 MB probablemente se beneficia
- Sirve para columnas con muchos valores distintos
- Admite filtrar por varias columnas

Partitioning, además

Cuando hace falta algo que el clustering no da

- Cada partición reuniría en promedio al menos 10 GB
- Se necesita conocer el costo antes de ejecutar
- Los datos se vencen, se borran o se reemplazan por período

Ninguno de los dos

Cuando no hay nada que descartar

- La tabla es chica: una tabla de métricas de pocos cientos de filas
- Nadie filtra por esas columnas

Las tablas del recorrido que sigue están muy por debajo de esos 10 GB por partición. Aun así, vamos a ver cuánto baja la lectura.

Recorrido · la misma query, dos tablas

Dos tablas con el mismo contenido, una particionada y otra no

La misma query de tres días sobre cada una

Qué dice el estimador antes de correr

Dónde se leen los bytes realmente facturados

Recorrido de partitioning · paso 1

Una tabla con partitioning y clustering

Enero de 2022, copiado dos veces: una tabla sin partitioning y esta. Las dos tienen 2.463.900 filas.



1 Particionada por día

Sobre la columna pickup_datetime.

2 Cada partición vence a los 60 días

Lo impone el sandbox, y se cuenta desde la fecha de la partición, no desde la carga.

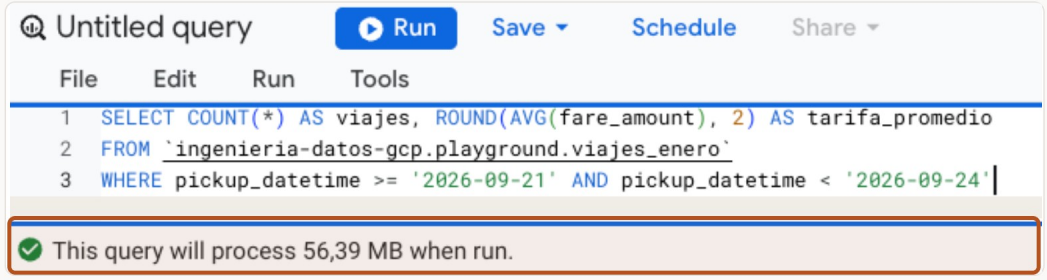
3 Clustering por zona

Dentro de cada día, las filas quedan ordenadas por pickup_location_id.

Recorrido de partitioning · paso 2

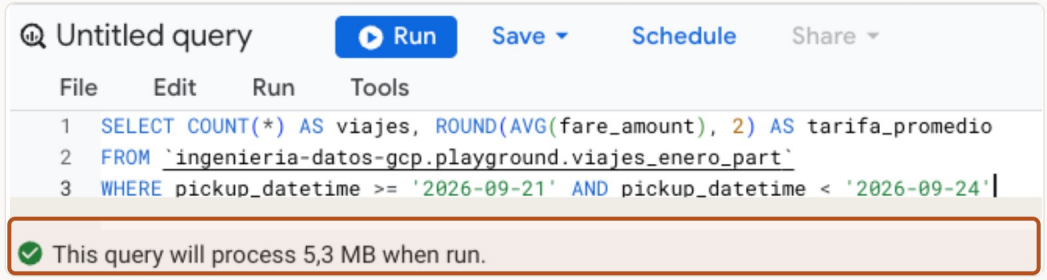
La misma query, las dos tablas

Viajes y tarifa promedio entre el 21 y el 23 de septiembre. Ninguna de las dos queries se ejecutó.



56,39 MB

Sin partitioning: lee la fecha y la tarifa del mes entero.



5,3 MB

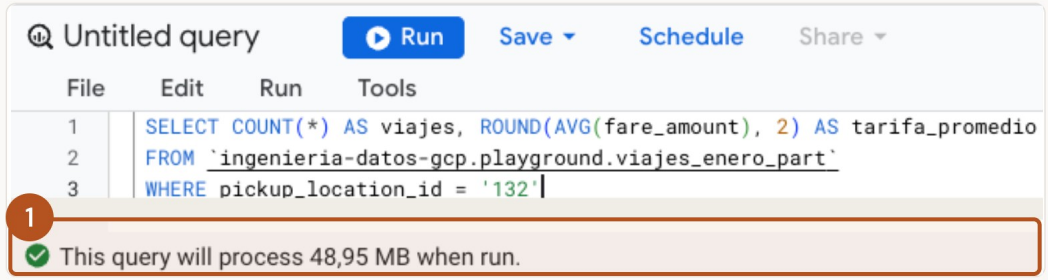
Con partitioning: lee 3 particiones de 31, diez veces menos.

El partition pruning ocurre antes de leer, y por eso el estimador ya lo descuenta.

Recorrido de partitioning · paso 3

El clustering se ve después de ejecutar

Filtrando solo por zona, antes de ejecutar



Estimado

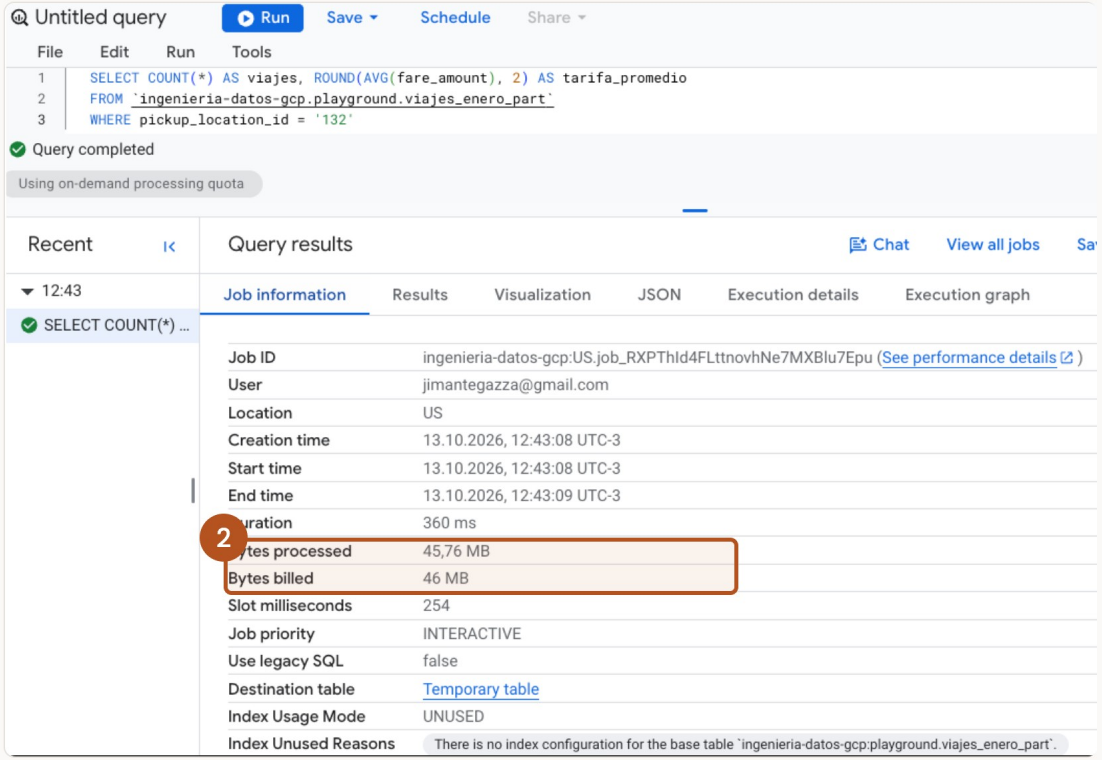
48,95 MB

Facturado

46 MB

El ahorro es chico porque cada partición pesa unos 4 MB y casi no hay bloques para descartar.

Después de ejecutar



Dónde se decide el gasto

En cuatro momentos distintos. Las decisiones tempranas son las que más ahorran: una tabla bien diseñada abarata todas las queries que vienen después.

1

Al diseñar la tabla

Según cómo se la va a consultar

- Partitioning por la columna con la que se filtra siempre
- Clustering por las que se filtran después
- Exigir el filtro de partición en cada query
- Tablas agregadas para lo que se consulta seguido

2

Al escribir la query

Cada vez que alguien pregunta

- Nombrar las columnas, nunca SELECT *
- Filtrar por la columna de partición
- Mirar el estimador antes de ejecutar

3

Al configurar el proyecto

Una vez, antes de que haga falta

- Elegir el modelo de contratación
- Tope de bytes por query
- Cuota diaria por proyecto o por usuario
- Alerta de presupuesto: avisa, no frena

4

Al operar

Todo el tiempo

- Revisar los procesos agendados y los tableros
- Buscar en el historial de jobs qué query factura más

Bajan lo que cuesta cada query

Ponen un techo y detectan desvíos

Google Cloud | The LAD

Esta noche sale la primera parte del TP

Parte 1

La capa raw

Escribir el esquema de los dos CSV y cargarlos en el dataset tp_raw. Lo confiable tipado, lo ambiguo como texto.

Recomendado

Mirar qué trajeron

Ninguno de los dos archivos está limpio. Recorrerlos con SQL ahora ahorra tiempo después.

La parte 2 (staging y marts con dbt) se publica más adelante. La entrega es una sola: miércoles 4 de noviembre, 23:59.

Clase 3 · Transformación y capa de métricas

20 de octubre, 19:00

Gracias

Preguntas

Google Cloud | The LAD